Synchronization

Conclusion

Definitions

Race condition: output of a concurrent program depends on the order of operations between threads

Mutual exclusion: only one thread does a particular thing at a time

- **Critical section**: piece of code that only one thread can execute at once

Lock: prevent someone from doing something

- Lock before entering critical section, before accessing shared data
- Unlock when leaving, after done accessing shared data
- Wait if locked (all synchronization involves waiting!)

Pre/Post Conditions

```
methodThatWaits() {
    lock.acquire();
    // Pre-condition: State is
  consistent

    // Read/write shared state

    while (!testSharedState()) {
         cv.wait(&lock);
   // WARNING: shared state may
   // have changed
   }

   // Read/write shared state
    lock.release();
}
```

```
methodThatSignals() {
    lock.acquire();
    // Pre-condition: State is
  consistent

    // Read/write shared state

    // If testSharedState is now
  true
    cv.signal(&lock);

    // NO WARNING: signal keeps lock

    // Read/write shared state
    lock.release();
}
```

Condition Variables Summary

- ALWAYS hold lock when calling wait, signal, broadcast
  - Condition variable is sync FOR shared state
  - ALWAYS hold lock when accessing shared state
- Condition variable is memoryless
  - If signal when no one is waiting, no op
  - If wait before signal, waiter wakes up
- Wait atomically releases lock
  - What if wait, then release?
  - What if release, then wait?

Condition Variables Summary, cont'd

- When a thread is woken up from wait, it may not run immediately
  - Signal/broadcast put thread on ready list
  - When lock is released, anyone might acquire it
- Wait MUST be in a loop

```
while (needToWait()) {
        condition.Wait(lock);
}
```

Synchronization Performance

- A program with lots of concurrent threads can still have poor performance on a multiprocessor:
  - Overhead of creating threads, if not needed
  - Lock contention: only one thread at a time can hold a given lock
  - Shared data protected by a lock may ping back and forth between cores
  - False sharing: communication between cores even for data that is not shared

Reducing Lock Contention

- Fine-grained locking
  - Partition object into subsets, each protected by its own lock
- Per-processor data structures
  - Partition object so that most/all accesses are made by one processor
  - Example: per-processor heap
- Ownership/Staged architecture
  - Only one thread at a time accesses shared data
  - Example: pipeline of threads

Best Practices

1. Consistent structure
2. Always synchronize with locks and condition variables
3. Always acquire the lock at the beginning of a method and release it right before the return
4. Always hold the lock when operating on a condition variable
5. When using condition variable - always wait in a while loop
6. Never use thread.sleep to have on thread wait for another thread

Synchronization without Lock

Communicating Sequential Processes (CSP/Google Go)

- A thread per shared object
  - Only thread allowed to touch object's data
  - To call a method on the object, send thread a message with method name, arguments
  - Thread waits in a loop, get msg, do operation
- No memory races!

Locks/CVs vs. CSP

- Create a lock on shared data

  = create a single thread to operate on data

- Call a method on a shared object

  = send a message/wait for reply

- Wait for a condition

  = queue an operation that can't be completed just yet

- Signal a condition

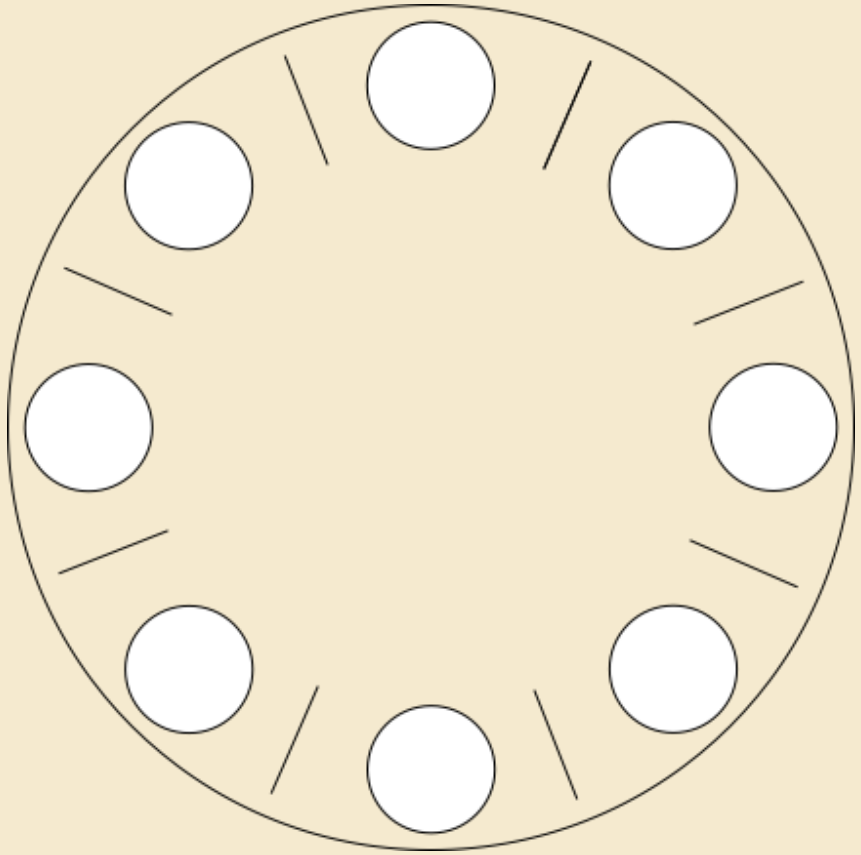
  = perform a queued operation, now enabled

Multi-Object Synchronization

Multi-Object Programs

- What happens when we try to synchronize across multiple objects in a large program?
  - Each object with its own lock, condition variables
- Performance
- Semantics/correctness
- Deadlock
- Eliminating locks

Dining Lawyers



Each lawyer needs two chopsticks to eat.

Each grabs chopstick on the right first.

Deadlock Definition

- Resource: any (passive) thing needed by a thread to do its job (CPU, disk space, memory, lock)
  - Preemptable: can be taken away by OS
  - Non-preemptable: must leave with thread
- Starvation: thread waits indefinitely
- Deadlock: circular waiting for resources
  - Deadlock => starvation, but not vice versa

Example: two locks

Thread A

```
lock1.acquire();
lock2.acquire();
lock2.release();
lock1.release();
```

Thread B

```
lock2.acquire();
lock1.acquire();
lock1.release();
lock2.release();
```

Two locks and a condition variable

Thread A

```
lock1.acquire();
…
lock2.acquire();
while (need to wait) {

  condition.wait(lock2);
}
lock2.release();
…
lock1.release();
```

Thread B

```
lock1.acquire();
…
lock2.acquire();
…
condition.signal(lock2);
…
lock2.release();
…
lock1.release();
```

Necessary Conditions for Deadlock

- Limited access to resources
  - If infinite resources, no deadlock!
- No preemption
  - If resources are virtual, can break deadlock
- Multiple independent requests
  - "wait while holding"
- Circular chain of requests

Preventing Deadlock

- Exploit or limit program behavior
  - Limit program from doing anything that might lead to deadlock
    - Provide enough resources
      - How many chopsticks are enough?
    - Eliminate wait while holding
      - Release lock when calling out of module
      - Banker's algorithm
    - Eliminate circular waiting
      - Lock ordering: always acquire locks in a fixed order
      - Resource allocation graph

Preventing Deadlock

- Predict the future
  - If we know what program will do, we can tell if granting a resource might lead to deadlock
  - Require experience

Deadlock Detection and Recovery

- Detection
  - Check for the cycle in the Resource Allocation Graph
  - Use Wait-For Graph Algorithm
- Recovery
  - Thread rollback: roll back or undo the lock.acquire
  - Thread restarting: restart the thread that is in a deadlocked state